

Solution to Homework 9

Meshach Aruoriwo Ogbor,
Md Rasel

December 13, 2023

1 Exercise 9.1 - Advanced MPI

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <string.h>
4 #include <mpi.h>
5
6 #define kappa 0.1
7
8 void read_matrix(const char *filename, int *M, int *N,
9                 double **local_A, int *local_rows, int rank, int
10                  size);
11
12 void write_matrix(const char *filename, double *
13                  local_A, int local_rows, int N, int rank, int size)
14 ;
15
16 void smear_matrix(double *local_A, double *local_B,
17                  int local_rows, int N, int rank, int size, MPIComm
18                  comm);
19
20 int main(int argc, char *argv[]) {
21     int rank, size;
22     MPI_Init(&argc, &argv);
23     MPI_Comm_rank(MPLCOMM_WORLD, &rank);
24     MPI_Comm_size(MPLCOMM_WORLD, &size);
25
26     int M, N;
27     double *local_A;
28     int local_rows;
```

```

21
22 // Load matrix from file and get dimensions
23 read_matrix("/work/korzec/LAB2/ex9/matrix.bin", &M
24             , &N, &local_A , &local_rows , rank, size);
25
26 // Perform smearing iterations
27 int iterations[] = {5, 15, 80};
28 int iter = 0;
29 for (int i = 0; i < 3; i++) {
30     for (int iter = 0; iter < iterations[i]; iter
31         ++) {
32         double *local_B = (double *)malloc(
33             local_rows * N * sizeof(double));
34         smear_matrix(local_A , local_B , local_rows ,
35                     N, rank, size, MPLCOMM_WORLD);
36         // Copy result back to A for the next
37         iteration
38         memcpy(local_A , local_B , local_rows * N *
39             sizeof(double));
40         free(local_B);
41     }
42     // Write the final smeared matrix to file
43     iter += iterations[i];
44     char outFile[50];
45     sprintf(outFile , "output_%d.bin", iter);
46     write_matrix(outFile , local_A , local_rows , N,
47                 rank, size);
48 }
49
50 free(local_A);
51
52 MPI_Finalize();
53
54 return 0;
55 }
56
57 void read_matrix(const char *filename , int *M, int *N,
58                 double **local_A , int *local_rows , int rank, int
59                 size) {
60     MPI_File fh;

```

```

52 MPI_Datatype subblock;;
53 MPI_Status status;
54
55 MPI_File_open(MPLCOMM_WORLD, filename ,
56               MPLMODE_RDONLY, MPI_INFO_NULL, &fh);
57
58 if (rank == 0) {
59     // Read M and N from file by rank 0
60     MPI_File_read(fh, M, 1, MPI_INT,
61                   MPI_STATUS_IGNORE);
62     MPI_File_read(fh, N, 1, MPI_INT,
63                   MPI_STATUS_IGNORE);
64 }
65
66 // Broadcast M and N to all processes
67 MPI_Bcast(M, 1, MPI_INT, 0, MPLCOMM_WORLD);
68 MPI_Bcast(N, 1, MPI_INT, 0, MPLCOMM_WORLD);
69
70 // Calculate local rows for each process
71 *local_rows = *M / size;
72
73 // Allocate memory for the local matrix
74 *local_A = (double *)malloc((*local_rows) * (*N) *
75                               sizeof(double));
76 int sizes[2], lsizes[2], starts[2];
77
78 // Only one dimension (2D matrix)
79 sizes[0] = *M;
80 sizes[1] = *N;
81 lsizes[0] = *M / size; // local_rows
82 lsizes[1] = *N;
83 starts[0] = rank * (*M) / size;
84 starts[1] = 0;
85
86 MPI_Type_create_subarray(2, sizes, lsizes, starts,
87                           MPIO_ORDER_C, MPLDOUBLE, &subblock);
88 MPI_Type_commit(&subblock);
89 MPI_File_set_view(fh, 2 * sizeof(int), MPLDOUBLE,
90                   subblock, "native", MPI_INFO_NULL);
91 MPI_File_read_all(fh, *local_A, (*local_rows) * (*

```

```

      N), MPI.DOUBLE, &status);
86
87     MPI_File_close(&fh);
88 }
89
90 void write_matrix(const char *filename, double *
    local_A, int local_rows, int N, int rank, int size)
    {
91     MPI_File fh;
92     MPI_Datatype subblock;
93     MPI_Status status;
94
95     int sizes[2], lsizes[2], starts[2];
96     int M = local_rows * size;
97     // Only one dimension (2D matrix)
98     sizes[0] = M;
99     sizes[1] = N;
100    lsizes[0] = local_rows; // local_rows
101    lsizes[1] = N;
102    starts[0] = rank * local_rows;
103    starts[1] = 0;
104    MPI_Type_create_subarray(2, sizes, lsizes, starts,
        MPI_ORDER_C, MPI.DOUBLE, &subblock);
105    MPI_Type_commit(&subblock);
106    MPI_File_open(MPI_COMM_WORLD, filename,
        MPI_MODE_WRONLY | MPI_MODE_CREATE,
        MPI_INFO_NULL, &fh);
107    MPI_File_write_at(fh, 0, &M, 1, MPI_INT, &status);
108    MPI_File_write_at(fh, 1 * sizeof(int), &N, 1,
        MPI_INT, &status);
109    MPI_File_set_view(fh, 2 * sizeof(int), MPI.DOUBLE
        , subblock, "native", MPI_INFO_NULL);
110    MPI_File_write_all(fh, local_A, local_rows * N,
        MPI.DOUBLE, &status);
111
112
113    MPI_File_close(&fh);
114 }
115
116 void smear_matrix(double *local_A, double *local_B,

```

```

117     int local_rows, int N, int rank, int size, MPIComm
118     comm) {
119     MPI_Request reqs[4];
120     int prev_rank = (rank - 1 + size) % size;
121     int next_rank = (rank + 1) % size;
122
123     double *temp_recv_top = (double *)malloc(N *
124     sizeof(double));
125     double *temp_recv_bottom = (double *)malloc(N *
126     sizeof(double));
127
128     // Start non-blocking send and receive for row
129     exchange
130     MPI_Isend(&local_A[0], N, MPLDOUBLE, prev_rank,
131     0, comm, &reqs[3]);
132     MPI_Irecv(temp_recv_bottom, N, MPLDOUBLE,
133     next_rank, 0, comm, &reqs[0]);
134     MPI_Isend(&local_A[(local_rows - 1) * N], N,
135     MPLDOUBLE, next_rank, 1, comm, &reqs[2]);
136     MPI_Irecv(temp_recv_top, N, MPLDOUBLE, prev_rank,
137     1, comm, &reqs[1]);
138
139     // Compute rows while communication is ongoing
140     for (int i = 1; i < local_rows - 1; i++) {
141         for (int j = 0; j < N; j++) {
142             int idx = i * N + j;
143             local_B[idx] = (1.0 / (1.0 + 4.0 * kappa))
144                 * (local_A[idx] + kappa * (local_A[(i -
145                 1) * N + j] + local_A[(i + 1) * N + j]
146                 + local_A[i * N + (j + 1) % N] +
147                 local_A[i * N + (j - 1 + N) % N]));
148         }
149     }
150
151     // Wait for the communication to finish
152     MPI_Waitall(4, reqs, MPI_STATUSES_IGNORE);
153
154     // Compute boundary rows i 0 and m - 1
155     for (int j = 0; j < N; j++) {
156         int i = 0;

```

```

144         int idx0 = i * N + j;
145
146         local_B[idx0] = (1.0 / (1.0 + 4.0 * kappa)) *
            (local_A[idx0] + kappa * (temp_recv_top[j]
            + local_A[(i + 1) * N + j] + local_A[i * N
            + (j + 1) % N] + local_A[i * N + (j - 1 + N
            ) % N]));
147         i = local_rows - 1;
148         int idx1 = i * N + j;
149         local_B[idx1] = (1.0 / (1.0 + 4.0 * kappa)) *
            (local_A[idx1] + kappa * (local_A[(i - 1) *
            N + j] + temp_recv_bottom[j] + local_A[i *
            N + (j + 1) % N] + local_A[i * N + (j - 1
            + N) % N]));
150     }
151
152 }

```

For compilation on Stromboli, we used the following command:

```
mpicc -o ex9 ex9.c
```

2 Submit Script for exercise 9.1

```
1 #!/bin/bash
2 #SBATCH --nodes=1
3 #SBATCH --ntasks=16
4 #SBATCH --partition=compute2011
5 #SBATCH --exclusive
6
7 module load mpi/openmpi/4.1.0
8 mpirun ./ex9
```

3 Results

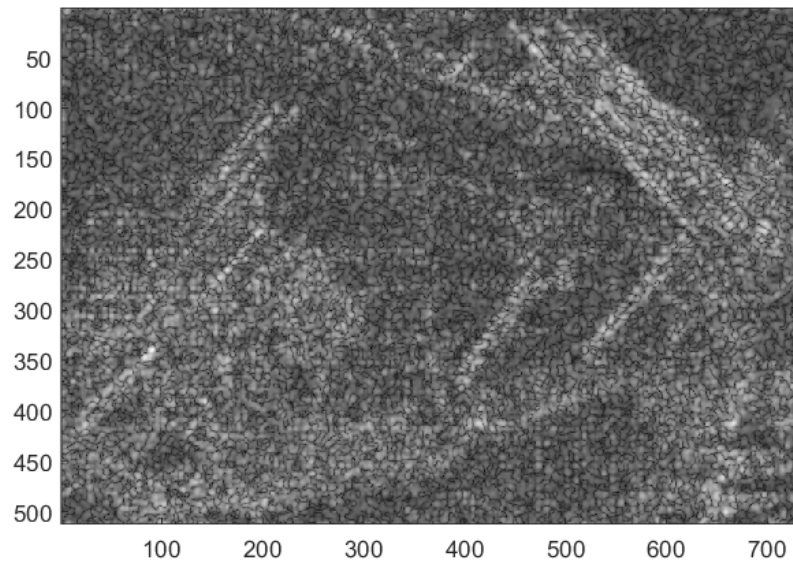


Figure 1: Image of iteration=5

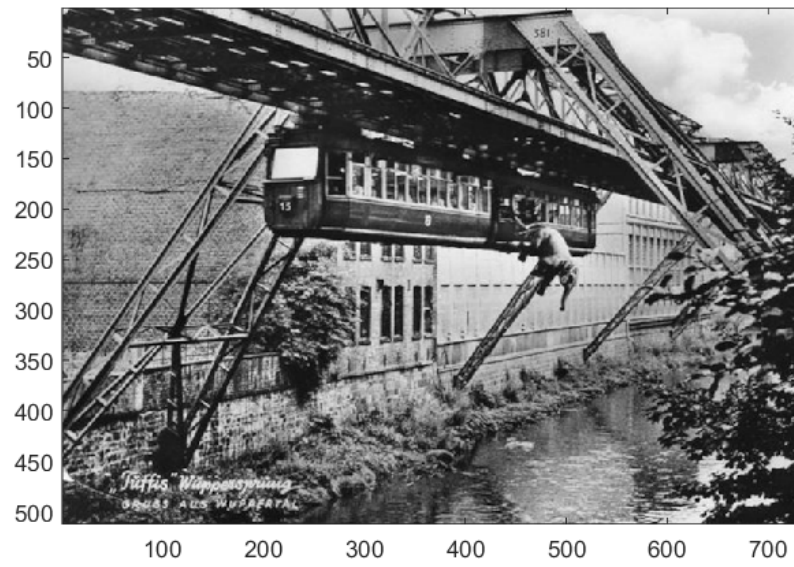


Figure 2: Image of iteration=20

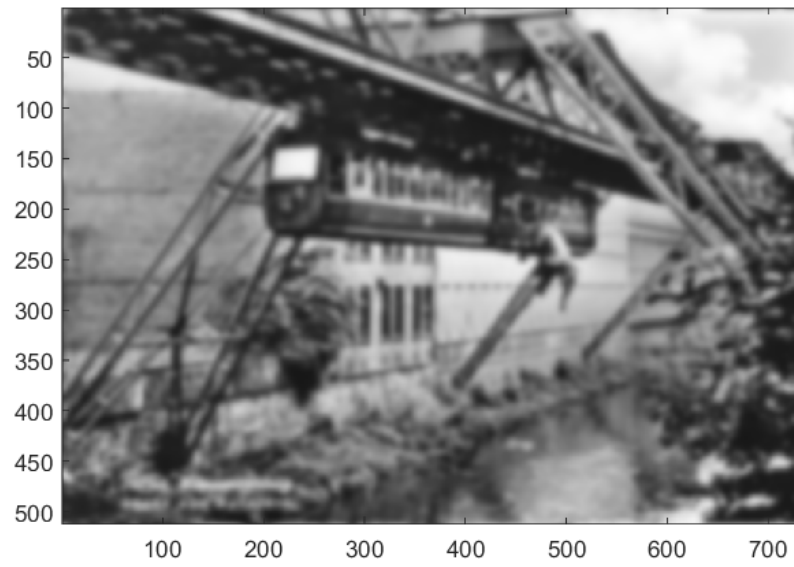


Figure 3: Image of iteration=100